IRCOMP GUI design report

ITU Phase I

Author's note: I developed this project individually, with approval from the subject guarantor. The authorship listed on the title page applies to all sections of this report and to my project implementation.

Project information	Value
Project	IRCOMP-GUI
Author and captain	Lukas (Viridis) Dubsik, xdubsil00
Exercise term	Monday 14:00–16:00
Work arrangement	Individual project, approved by the subject guarantor
Authored scope	Prepare, Review and Export views, shared client state, Python services and storage
Repository	https://github.com/viridisquercus/IRCOMP-GUI
Source snapshot	379f4bfb16f79db650c0d9c32b6508c68471f34e

Project topic

IRCOMP-GUI is the first interface for IRCOMP, developed as my project in ITU at FIT. It enables the user to prepare a CRISPR analysis request and then go through the results supplied by an analysis tool. The request contains genomic intervals (targets), while the results contain candidates that the user can put into collections and write decisions about.

The application runs locally, with the data and operations written in Python (FastAPI) and the interface in React with TypeScript. Saved studies are stored in SQLite. The scientific computation itself is not implemented here yet. The application prepares its input and shows the imported output, while the search and scoring belong to IRCOMP.

Contents

1 User research
2 Competing solutions
3 Interface design
4 GUI architecture and API
5 Source submission
Appendix Source files and references

The images below are the original wireframes, with made-up data. They are not screenshots of the finished application.

1 User research

1.1 Research record

For the research, I discussed the project with Vojtěch Bartoň and Vojtěch Bystrý from Masaryk University. This was done through interviews and longer discussions, including the current dummy version of the interface. Both liked it and agreed with the direction of the design. The application will still need more functionality, as this version only covers part of IRCOMP.

Participant	Role and affiliation	**Method**
Ing. Vojtěch Bartoň	Analyst, Masaryk University [6]	In-person individual interview and follow-up discussions
Mgr. Vojtěch Bystrý, Ph.D.	Head of Bioinformatics Core Facility, CEITEC MU [7]	In-person individual interview and follow-up discussions

I interviewed both participants in person, separately. I brought the first version of the prototype and went through it with them while they pointed out problems. Two more sessions followed, until the most serious issues raised during the discussions had been fixed.

Based on this feedback, I kept the current version as the basis for further development. This was a discussion of the design, without measuring how long tasks take or how often the user completes them.

1.2 Concrete user needs

Bartoň mainly asked for clear and easy setup. Using the application should not depend on knowing its internals or having someone experienced configure it. He also pointed out that bioinformaticians may not have the same depth of biological knowledge as their colleagues, so the inputs and settings need to be understandable. Safe handling of research data was another important point. Connections to databases and other tools were discussed as a later addition.

Bystrý raised similar points, but put more emphasis on the analytical depth. He saw the current implementation as a good starting point and demonstrator, while a tool used for real work would need more advanced capabilities. These are left for later development. The interface also needs to support people outside bioinformatics, with different biological and computational experience. Keeping the basic workflow simple should still leave room for advanced settings.

1.3 Interpretation of the feedback

Based on these discussions, I kept the current interface as the basis for further development. The immediate priorities are an understandable setup and workflow, clear inputs and safe handling of data. Advanced analysis and connections to databases and other tools belong to the later IRCOMP integration. The current revision keeps unfinished edits separate from accepted data and retains the source of imported evidence. These help protect the work from accidental changes, but do not by themselves establish data security.

2 Competing solutions

2.1 Basis of the comparison

For comparison, I chose CRISPOR and CHOPCHOP. Both are tools for selecting CRISPR guides and inspecting their predicted properties. I used their official documentation and the CRISPOR paper [1–3]. Unlike the current GUI, both also run the analysis. Here, I am comparing how the user enters the request and works with the results, without performing separate usability testing of these applications.

The discussions pointed to a simple entry into the application, understandable settings and enough depth for later work. I therefore looked at how each tool handles input, separates basic and advanced options, explains its scores and lets users keep results. I also checked whether the reference context stays clear. Both tools offer a browser interface, which reduces the initial setup needed to try them. Keeping the data local is a different choice in IRCOMP-GUI, so its installation still needs to be straightforward.

2.2 CRISPOR

CRISPOR accepts a sequence together with the selected genome and PAM. It then lists the possible guides and their predicted scores [1, 2].

What works well: (1) specificity and efficiency are shown separately, so the user can compare both; (2) warnings point out problems with the input or guides, such as unusual GC content; (3) the results can be downloaded, so they can be shared or kept outside the website.

What needs care: (1) cDNA input is a common mistake mentioned in the manual, even though the sequence itself may look valid; (2) the genome used last becomes the default, so it needs to be checked before another experiment; (3) a guide can be specific but inefficient, or the opposite, so the first one in a ranking is not automatically the best choice.

For my application, I kept the reference visible and the scores separate. This lets the user see what the request refers to and what each score actually describes. The GUI only checks the supported metadata and coordinates, it does not perform the sequence checks done by CRISPOR.

2.3 CHOPCHOP

CHOPCHOP accepts a gene identifier, genomic coordinates or a sequence. The results are shown in an interactive table along with a view of the target region [3].

What works well: (1) the user can start with different types of input; (2) the region view shows where the candidates are located; (3) the basic settings are separate from the advanced options, so the user does not need to go through every setting at the start.

What needs care: (1) incompatible scoring settings can produce zero values, which may look like valid low scores; (2) options are remembered and can remain selected after changing the targeting mode; (3) G20 is offered as an efficiency option, but only checks for a G at a given position, rather than using a predictive model.

Because of this, the GUI keeps the availability of a score separately from its value. A missing score is shown as missing, while a supplied zero stays zero. The active context also stays visible. Separating basic and advanced options is useful for the future capabilities requested by Bystrý. Direct import of CRISPOR or CHOPCHOP output is not implemented yet.

3 Interface design

3.1 Structure and data

The interface is divided into three views, Prepare, Review and Export. All of them work with the same open study. Its title, selected revision and save state stay visible, so the user can move between the views without losing the context of the work.

View	Data and operations
Prepare	Reference, profile and target intervals. Add, edit or remove a target, change context and run local checks.
Review	Imported candidates, their scores and source. Create collections, add or remove members and record decisions.
Export	Saved request or selected review collections. Preview the contents and download the prepared file.

Switching the view does not open another study. Selecting a different revision changes the targets, evidence and collections being shown. Older request values are kept, so evidence imported for them can still be inspected. The decisions are stored inside a collection belonging to that revision.

3.2 Authored interface

I developed all three views together, as the individual parts work with the same study. Below are the original wireframes used for the design. Some of the texts changed during implementation, and all the values shown in them are made up.

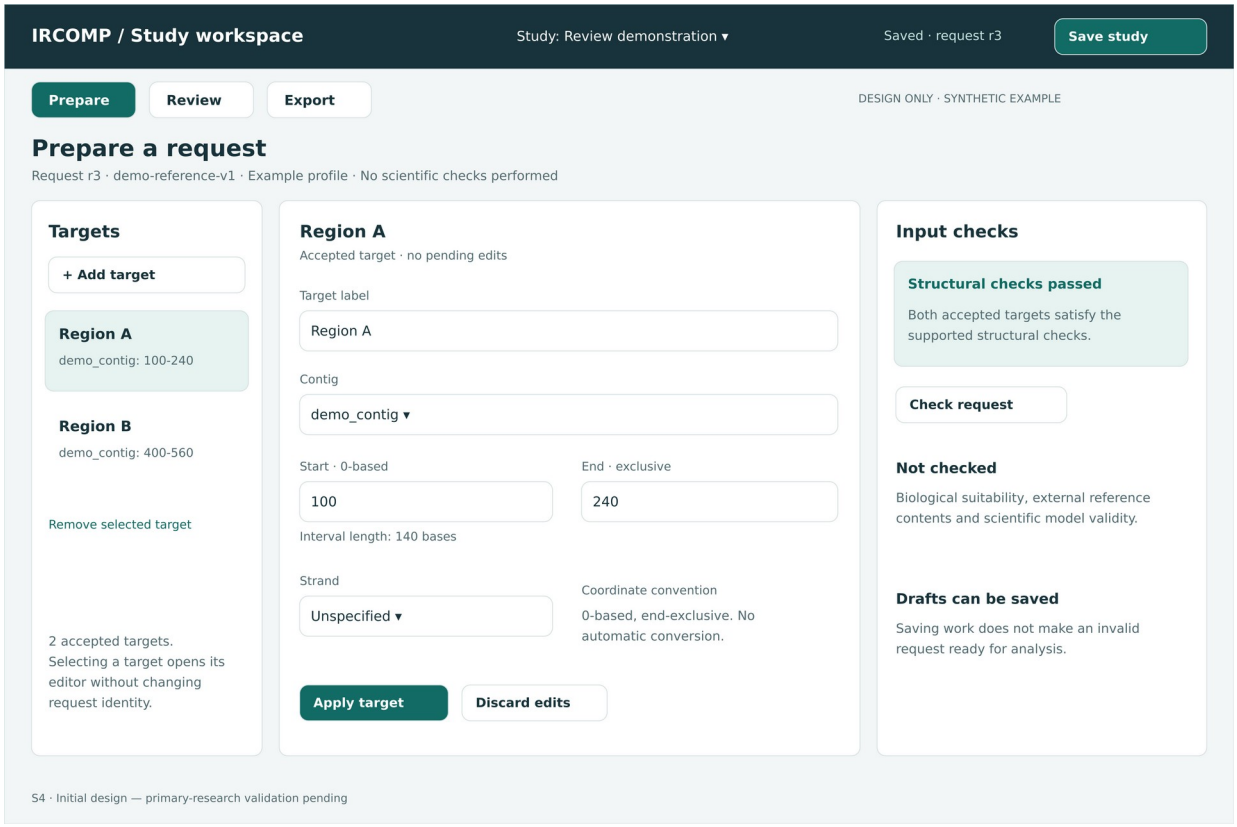


Figure 1. Prepare wireframe. Target list on the left, inline editor in the centre and request context and checks on the right.

3.2.2 Review interface

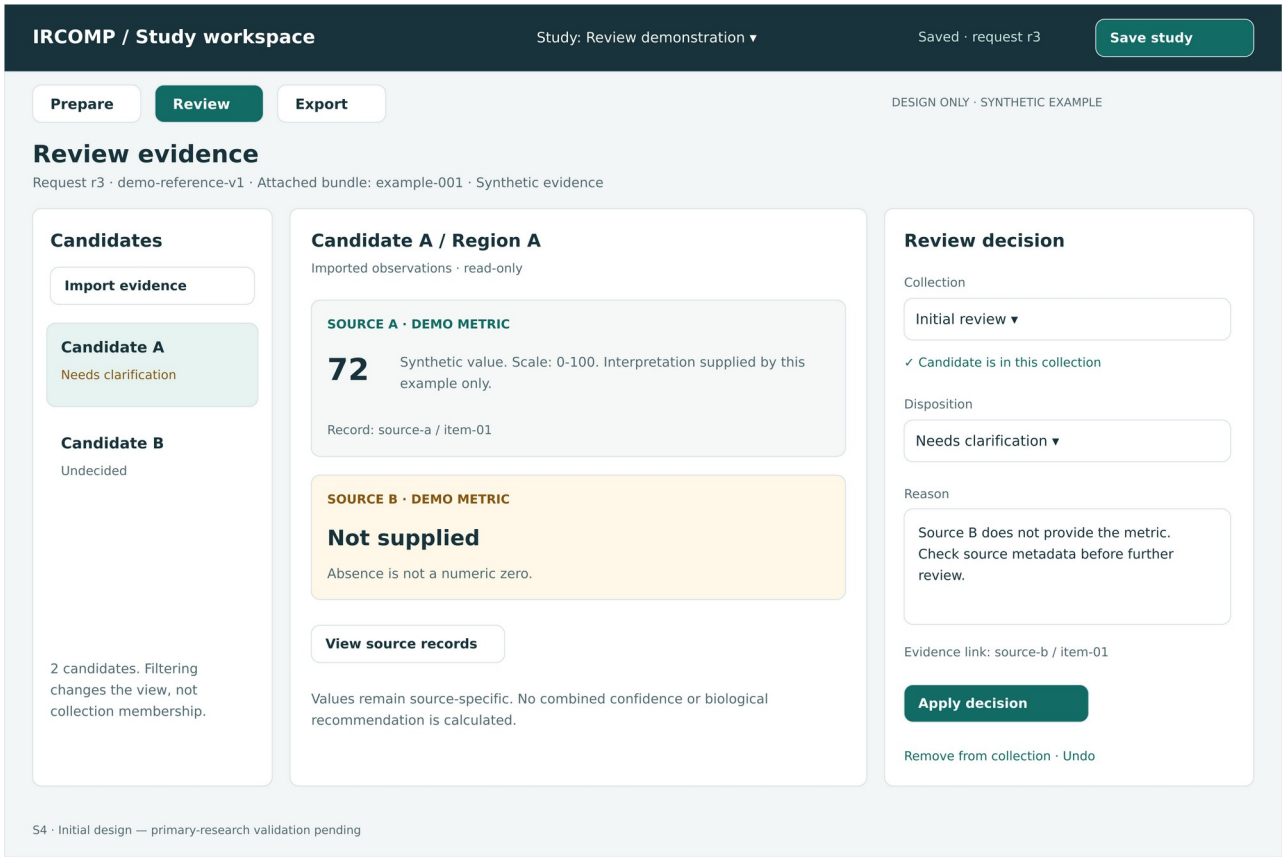


Figure 2. Review wireframe. Candidates, imported evidence and the decision editor appear together.

The user selects a candidate and adds it to a collection. They can then choose a disposition and write the reason for it. One candidate may be in multiple collections, with a different decision in each. The imported evidence itself is never changed by this.

The middle panel shows the scores provided by the tool, including the model, scale, direction and whether a value was supplied at all. The original record and information about its source can also be opened here.

I put the evidence beside the decision editor so the user can refer to it while writing the reason. Whether this layout is easy to use during actual tasks still needs to be checked in usability testing.

Interaction scenario

To go through the review, the user imports an evidence file, checks its preview and attaches it. They then add a candidate to a collection, write a reason and apply the decision. The membership and number of unresolved decisions update from the returned study. Save writes the study to SQLite.

3.2.3 Prepare and export interactions

In Prepare, the list of targets is on the left, beside the editor. The coordinates are 0-based and the end is not included, so 100 to 200 means 100 bases. Apply changes the open study, while Save writes it to the disk. Changing the accepted coordinates creates a new request revision. The previous evidence stays with the revision it was made for.

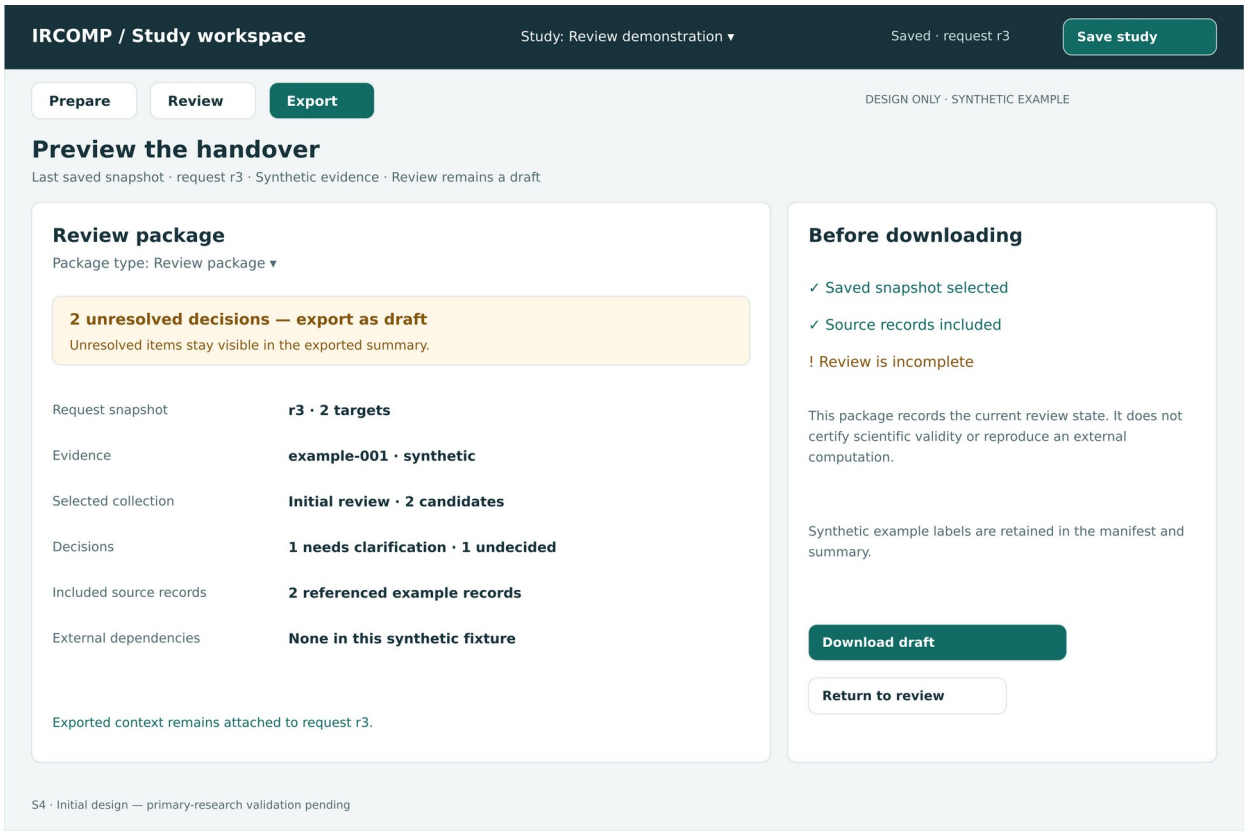


Figure 3. Export wireframe. The selected saved context and the contents of the handover remain visible before download.

Export always uses saved data. The user chooses either the request or the review package, checks the preview and downloads the file. A review package contains the selected collections and the original evidence files. If there are unfinished decisions, it is marked as a draft.

When the input is wrong, the application shows what is wrong and, where available, which field it belongs to. The states use text along with colours, and on a narrow screen the panels move under each other. Accessibility and usability testing have not been completed yet.

4 GUI architecture and code

4.1 Overall architecture

The application is split between the React interface and the Python model and services. The views show the current state and receive user input. The controller sends the operation to FastAPI, which calls the appropriate service. The services use the domain rules to check the change. Storage and file reading are accessed through interfaces, so the same services can be used without a browser.

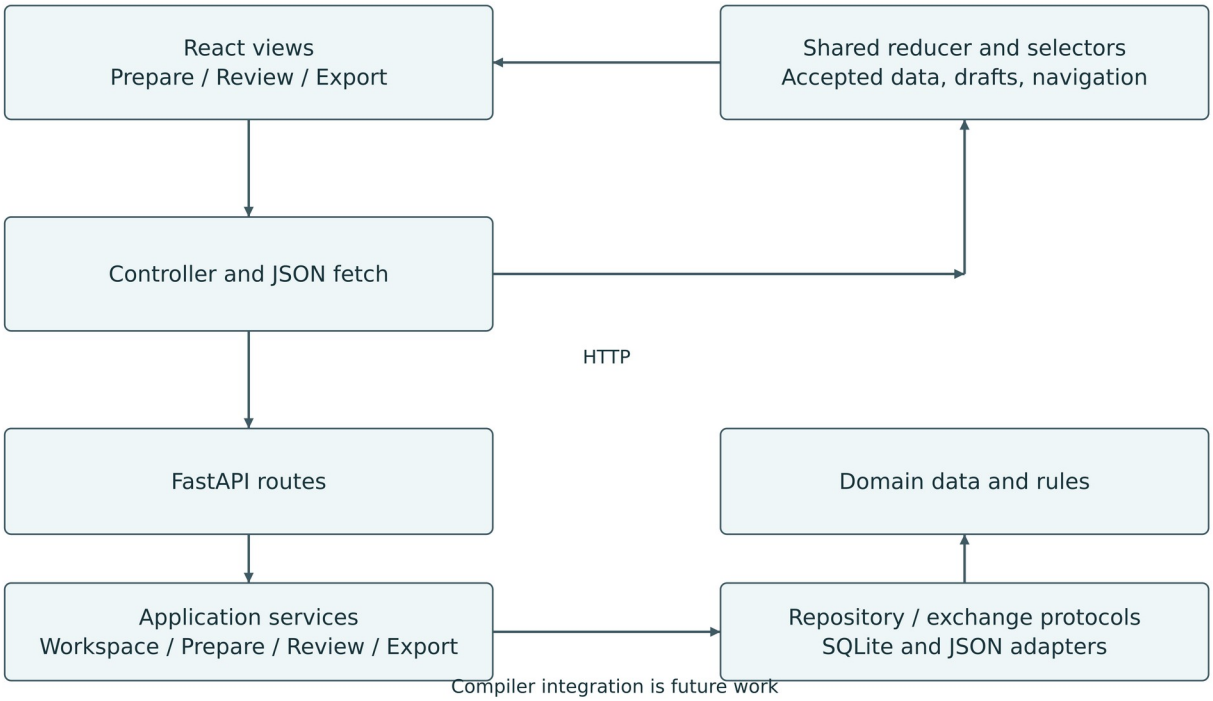


Figure 4. Repository architecture diagram. Services use repository and exchange interfaces. The shared reducer supplies the React views.

Model	Purpose
Study and Revision	Study identity and retained request contexts. A revision stores its reference, profile and targets.
EvidenceBundle and Candidate	Imported results with their exact request context, source and metric information.
Collection and Membership	Candidate selection and a separate decision, rationale and supporting references.
TargetDraft and DecisionDraft	Unapplied editor input, kept apart from the accepted request and decisions.

The candidate ID is only unique inside its bundle. Because of this, the application identifies a candidate using both the bundle ID and candidate ID. The original evidence bytes and their checksums are kept separately from the user’s decisions.

4.2 My implementation and API

My part covers Prepare, Review and Export, along with the shared client state and the Python operations they use. The individual source files are listed in the appendix.

Operation	HTTP interface under /api/v1
Catalogue and saved studies	GET /catalogue; GET /studies
New, saved, duplicate or imported study	POST /workspaces
Read or close working copy	GET or DELETE /workspaces/{id}
Apply an editing operation	POST /workspaces/{id}/commands
Save and one-step Undo	POST /workspaces/{id}/save; POST /workspaces/{id}/undo
Check a request	POST /workspaces/{id}/validate
Inspect or cancel an import	POST /imports; DELETE /imports/{id}
Prepare and download an export	POST /exports; GET /exports/{id}/file

Each command has a type field saying which operation to perform, together with the values needed for it. Prepare supports rename_study, apply_target, remove_target, rename_target and set_context. Review supports attach_evidence, create_collection, rename_collection, remove_collection, add_member, remove_member and set_decision.

State and update flow

The application data are stored in the Python model. The client has its shared state in a reducer, including the selected items and unfinished editor text. The study is not stored in the DOM elements. After an operation, the controller passes the returned workspace to the reducer and all related parts of the interface update. The page itself does not reload.

Every change of the editor text also increases its generation counter. If a response arrives after the user has already typed something newer, that text is kept. The target and decision drafts are saved separately from the accepted values. Other unfinished input, such as a title not renamed yet or fields of a context change, is not saved this way.

FastAPI checks the responses against their models before sending them. An error has its code, message and diagnostics, which may also point to the wrong field. The exchange schema, TypeScript types and numeric limits are generated from the Python definitions by scripts/generate_contracts.py, so they do not have to be maintained separately.

5 Source submission

The source archive includes the Python model and operations, the working GUI, contracts, examples and documentation. I worked on the project alone, with the AI assistance described below. External libraries keep their own licences.

Build and run

The project requires Python 3.12 or newer and Node 20.19 or newer. To install and run it, use make install, then make build and make run from the repository root. The application is then available at http://127.0.0.1:8000. The README also lists the same commands without Make.

For the checks, first run make install-dev. make check checks the generated files, Python linting and typing, TypeScript and formatting. make test builds the client and runs the Python and Cypress tests. These check the functionality. Testing whether people can use the interface is a separate part.

Archive and final handover

make pack creates release/ircomp-gui-source.zip and its checksum. It needs a clean, committed checkout. For submission, the archive needs to be named 00_xdubsil00_source.zip and this report 00_xdubsil00_navrh.pdf.

Current limitations

The compiler, workflow graphs, YAML specification editing and running the analysis are still future work. The JSON format used here is separate from the IRCOMP specification language. Open studies live in service memory, so restarting the service loses unsaved work. The application should be used with one tab per study, as changes from multiple working copies are not merged.

The reviewed version still has issues with recovering decision drafts, old values left in the context editor and abandoned import previews. These remain to be addressed before the final handover.

AI assistance

I wrote the basis of the Cypress tests and the scenarios, then used AI to refine them and fill in missing parts. AI was also used for finding errors and inconsistencies and helping with the documentation. This report was drafted with AI assistance using the repository, my discussion notes and the information I provided.

Appendix Source files

Below is the list of source files, with paths relative to the repository root. This includes the package initializer files. The generated browser types and constants are made from the Python definitions by the generator listed here.

scripts/ generate_contracts.py

src/ircomp_gui/ __init__.py, __main__.py

src/ircomp_gui/adapters/ __init__.py, exchange.py, resources.py, sqlite.py

src/ircomp_gui/api/ __init__.py, app.py, requests.py

src/ircomp_gui/application/ __init__.py, commands.py, export.py, imports.py, ports.py, prepare.py, responses.py, review.py, workspace.py

src/ircomp_gui/domain/ __init__.py, catalogue.py, constants.py, errors.py, models.py, rules.py, serialization.py

tests/ test_contracts.py

web/ cypress.config.ts, index.html, vite.config.ts

web/cypress/e2e/ recovery.cy.ts, review.cy.ts, workspace.cy.ts

web/cypress/ helpers.ts

web/scripts/ test.mjs

web/src/app/ App.css, App.tsx, WorkspaceProvider.tsx, controller.ts, selectors.ts, state.ts

web/src/features/export/ ExportView.css, ExportView.tsx

web/src/features/prepare/ PrepareView.css, PrepareView.tsx

web/src/features/review/ ReviewView.css, ReviewView.tsx

web/src/ main.tsx

web/src/shared/ ImportPreview.css, ImportPreview.tsx, api.ts, constants.generated.ts, constants.ts, types.generated.ts, types.ts

web/src/styles/ base.css

References

[1] CRISPOR official manual. https://github.com/maximilianh/crisporWebsite/blob/master/manual/manual.md

[2] Concordet and Haeussler. CRISPOR: intuitive guide selection for CRISPR/Cas9 genome editing experiments and screens. https://pubmed.ncbi.nlm.nih.gov/29762716/

[3] CHOPCHOP official instructions. https://chopchop.cbu.uib.no/instructions

[4] IRCOMP-GUI source and documentation. https://github.com/viridisquercus/IRCOMP-GUI/tree/379f4bfb16f79db650c0d9c32b6508c68471f34e

[5] Supplied ITU project instructions, Phase I design report template and research/testing guide.

[6] Masaryk University profile of Ing. Vojtěch Bartoň. https://www.muni.cz/en/people/429992-vojtech-barton

[7] Masaryk University profile of Mgr. Vojtěch Bystrý, Ph.D. https://www.muni.cz/en/people/98640-vojtech-bystry